

Persistent Login – 3 Approaches Design Canvas

Persistent Login – 3 Approaches Design Document

1. Overview

This document describes three different approaches to implement Persistent Login (up to 12 hours) in the system, considering:

- Navitaire limitation (single token – 15 mins validity)
 - High user scale (B2B + B2C)
 - Performance and security considerations
-

2. Participants

- User
 - UI (React)
 - Backend (Node.js)
 - Navitaire
-

3. Key Constraint

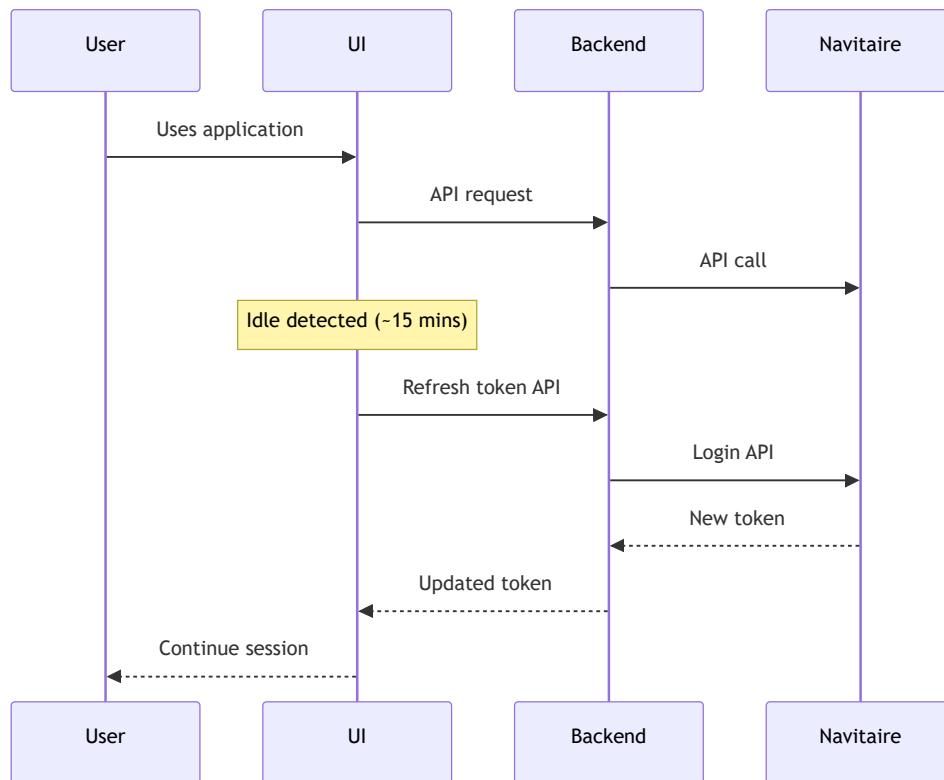
- Navitaire supports only one token
 - Token validity: 15 minutes
 - No access token / refresh token model
 - Token must be regenerated via login API
-

4. Approach 1: Background Token Refresh (Auto Refresh API)

Description

- System automatically calls refresh/login API after idle (~15 mins)
 - Keeps session alive continuously
 - Time-based approach
-

Sequence Diagram



Pros

- Simple implementation
- No credential handling required
- Seamless user experience
- Works with existing system

Cons

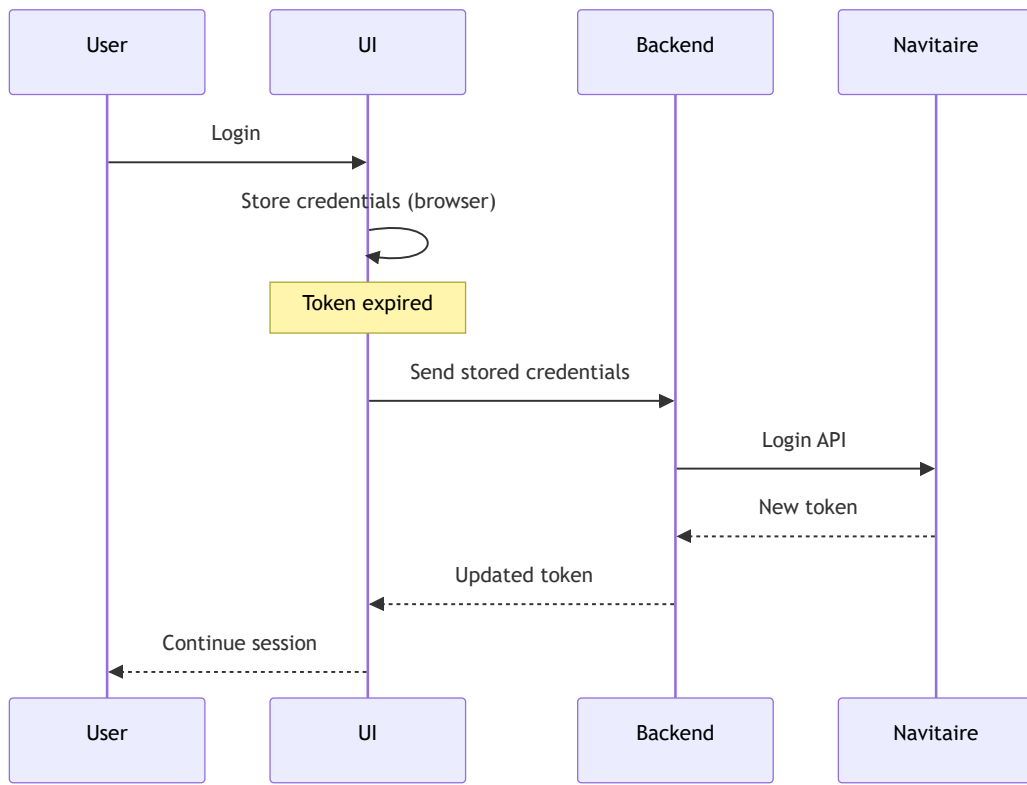
- Generates unnecessary API calls during idle
 - Not event-driven
 - Adds non-transactional traffic
 - Not optimal for large-scale systems
-

5. Approach 2: Store Credentials in Browser (Insecure Silent Login)

Description

- Store username/password in browser
- Re-login silently when token expires

Sequence Diagram



Pros

- No background API calls
- Simple re-login logic

Cons

- Highly insecure
 - Storing credentials in browser is risky
 - OTP cannot be handled securely
 - May require disabling OTP (security violation)
 - Not compliant with security standards
 - Not recommended
-

6. Approach 3: Middleware-Based Re-Authentication (Event-

Driven – Final Approach)

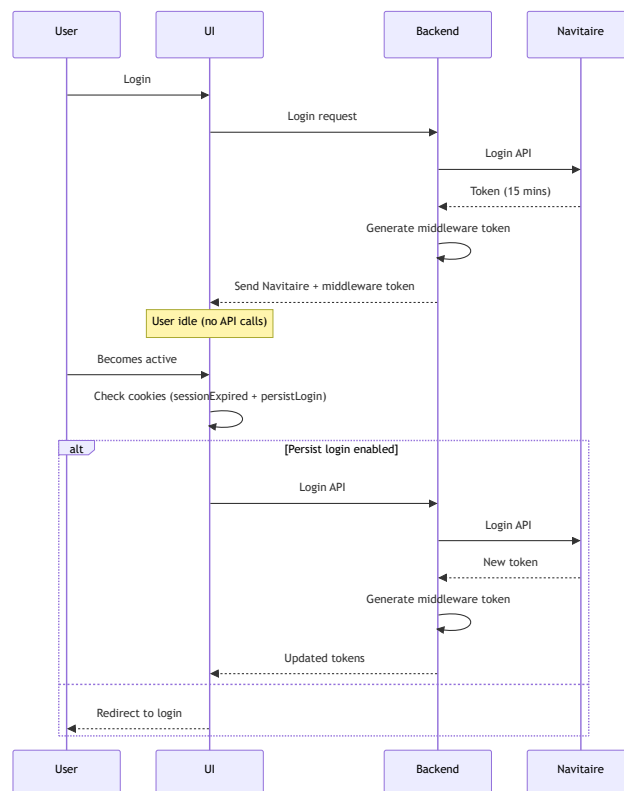
Description

- No API calls during idle
 - Re-login only when user becomes active
 - Uses backend-stored credentials + middleware token
-

Flow Overview

- Credentials stored securely in backend
 - Middleware token generated (encrypted: timestamp + browser info)
 - Token validated on every request
 - Re-login triggered only when needed
-

Sequence Diagram



Pros

- No unnecessary API calls during idle
- Event-driven (only when user is active)
- Highly scalable
- Reduced system load
- Better performance efficiency
- Secure (no credentials in browser)

Cons

- More complex implementation
- Requires secure backend credential storage
- Slight delay when user resumes
- Middleware validation required

7. Comparison of All Approaches

	Criteria	Approach 1: Auto Refresh	Approach 2: Browser Credentials	Approach 3: Middleware
1	API Calls	High (continuous)	Low	Very Low (event-based)
2	Performance	Medium	Good	Best
3	Scalability	Risk at scale	Moderate	High
4	Security	Good	Very Poor	Good
5	Idle Behavior	Calls API	No calls	No calls
6	Implementation	Easy	Easy	Complex
7	OTP Compliance	Yes	No	Yes
8	Recommended	No (at scale)	No	Yes

8. Final Recommendation

- Approach 1: Functional but not scalable due to unnecessary traffic
- Approach 2: Not acceptable due to security risks
- Approach 3: Best balance of performance, scalability, and security

Conclusion

The Middleware-Based Approach (Approach 3) is the most suitable solution:

- Eliminates unnecessary background calls
- Works within Navitaire limitations
- Handles large-scale users efficiently
- Maintains security without compromising OTP

End of Document